

AgentForge: A Role-Structured Multi-Agent Framework for Repository-Level Software Repair with Execution Feedback

Rajesh Kumar, Waqar Ali, Junaid Ahmed, Najma Imtiaz Ali, Shaban Usman

Abstract—Repository-level software repair requires more than generating a plausible patch: an agent must inspect a codebase, localize the defect, modify the repository, execute relevant checks, interpret failures, and revise the patch within a limited budget. We present AGENTFORGE, a role-structured multi-agent framework that separates planning, editing, test construction, debugging, and review while maintaining a shared repository state and execution history. The revised design introduces an explicit *post-edit execution invariant*: every state-changing edit must be followed by execution in the same repository environment before another edit is permitted. Agent-generated tests are treated as auxiliary debugging signals rather than ground-truth correctness evidence, and final correctness is measured only by the official SWE-BENCH evaluation harness. We specify a leakage-free protocol that excludes reference patches and benchmark grading metadata from agent-visible inputs, resets episodic memory for each evaluation instance, and compares agent configurations under matched model, tool, token, cost, and execution budgets. On the seeded 50-instance subset, the available official harness report for AGENTFORGE resolves 1/50 tasks (2.0%, exact 95% Clopper-Pearson CI: 0.05%–10.65%); matched control configurations currently provide generation-stage statistics but lack saved official grading reports, so no architectural superiority or controlled ablation effect is claimed. The framework and evaluation artifacts will be released with a frozen commit and task-level predictions.

Index Terms—Large language models, multi-agent systems, automated program repair, repository-level software engineering, execution feedback, SWE-bench.

I. INTRODUCTION

Large language models (LLMs) can synthesize code from natural-language specifications, but repository-level software repair remains difficult. A realistic repair task requires the system to interpret an issue report, inspect an unfamiliar repository, identify relevant files, modify existing code, execute tests, diagnose failures, and avoid regressions. Function-level benchmarks such as HumanEval emphasize isolated synthesis, whereas SWE-BENCH evaluates changes to real repositories under project-specific tests and environments [1, 2].

R. Kumar is with the International Research Center for Complexity Sciences, Hangzhou International Innovation Institute, Beihang University, Hangzhou 311115, China (e-mail: rajakumarlohano@gmail.com).

W. Ali is with the Department of Computer Science, College of Science, Mathematics and Technology, Wenzhou-Kean University, Wenzhou 325060, China (e-mail: waqar.uestc@yahoo.com).

J. Ahmed is with the Department of Computer Systems Engineering, Sukkur IBA University, Sindh, Pakistan (e-mail: j.bhatti@iba-suk.edu.pk).

N. I. Ali is with the Faculty of Information and Communication Technology, Universiti Teknikal Malaysia Melaka, Melaka 76100, Malaysia (e-mail: najma@utem.edu.my).

S. Usman is with the Yibin Park of the University of Electronic Science and Technology of China, Yibin 644000, China (e-mail: shabanusman@yahoo.com).

Tool-using software engineering agents address part of this challenge by exposing shell, editor, search, and execution interfaces to an LLM [3–5]. Multi-agent systems add role specialization, assigning planning, coding, testing, or review to separate agents [6–9]. These directions are complementary: tool access supplies interaction with the repository, while decomposition can make intermediate responsibilities explicit. However, claims about the value of execution or multi-agent structure require carefully controlled evaluation. A system with more calls, more tokens, more tools, and more test attempts cannot be fairly compared with a weaker single-call baseline without controlling those resources.

This paper studies a narrower question: *under matched computational and tool budgets, does a role-structured workflow with enforced post-edit execution improve repository-level repair?* We introduce AGENTFORGE, a framework with five roles: Planner, Coder, Tester, Debugger, and Critic. The central design feature is not merely that execution tools are available, but that the orchestrator treats execution as a state-machine constraint. After a Coder or Debugger action changes the repository, the system must execute a validation command and record its outcome before another state-changing action can occur.

The revised evaluation is designed around four validity requirements. First, the agent receives only the issue statement and the base repository checkout; reference patches, reference-derived file paths, benchmark test metadata, and grading labels are excluded. Second, execution occurs in the same instance-specific repository environment used during interaction, while final scoring is performed independently through the official SWE-BENCH harness. Third, generated tests are auxiliary hypotheses and are not equated with benchmark correctness. Fourth, comparisons are paired and budget controlled.

Contributions. This work makes the following contributions:

- A role-structured repository-repair architecture that separates planning, editing, auxiliary test generation, debugging, and review while preserving a single mutable repository state.
- An explicit post-edit execution invariant enforced by the orchestrator rather than delegated to a planner-generated sequence.
- A contamination-resistant evaluation protocol that excludes gold-patch information, isolates episodic memory, and uses the official SWE-BENCH evaluation infrastructure.

- A controlled experimental design that separates the effects of role decomposition, forced execution, tool access, and additional test-time computation under matched budgets.
- A reproduction artifact in which the reported tables are generated from run outputs rather than transcribed: the released code regenerates every results table, confidence interval, and paired test directly from the official harness reports and task-level ledgers, so a discrepancy between the artifact and the manuscript is a build failure rather than a matter of trust.
- A transparent partial evaluation on a seeded 50-instance subset: the available official harness report records 1/50 resolved tasks for AGENTFORGE, while ungraded controls and incomplete ablations are explicitly separated from official correctness claims.

The paper deliberately avoids claiming that execution-enabled agents are new. SWE-agent and OpenHands already support iterative interaction with executable environments [3, 4]. Our contribution is the orchestration policy, its controlled evaluation, and the separation of auxiliary runtime signals from hidden benchmark correctness.

II. RELATED WORK

A. Code Generation and Automated Program Repair

Large pretrained models such as Codex, AlphaCode, StarCoder, and Code Llama established strong code-generation capabilities [1, 10–12]. Automated program repair (APR), however, requires a candidate patch to satisfy behavioral constraints in an existing system. Classical APR systems search or synthesize patches using tests and fault-localization signals [13]. More recent LLM-based repair systems use error messages, execution traces, or iterative self-debugging to refine outputs [14–18]. These methods motivate the use of runtime evidence, but generated or visible tests can provide incomplete supervision and must not be conflated with hidden benchmark correctness.

B. Tool-Using Software Engineering Agents

ReAct interleaves reasoning and tool use, and Reflexion adds memory-based self-correction [19, 20]. SWE-agent provides an agent-computer interface for repository navigation, editing, and command execution [3]. OpenHands offers an open platform with sandboxed runtime interaction [4], while AutoCodeRover combines repository analysis with patch generation [5]. These systems demonstrate that executable environments are already central to autonomous software engineering. Accordingly, AGENTFORGE does not claim novelty for sandbox access itself; it studies an orchestrator-enforced execution schedule under controlled budgets.

C. Multi-Agent Software Engineering

MetaGPT and ChatDev model software development as communication among specialized roles [8, 9]. AutoGen provides a general framework for multi-agent interaction [21]. MAGIS applies manager, repository-custodian, developer, and QA roles to GitHub issue resolution [6]; AgentMesh similarly

explores cooperative role decomposition [7]. Role specialization can improve modularity and interpretability, but additional agents also increase test-time computation. A convincing evaluation must therefore compare role structures while holding the base model, tool set, context, and resource budget constant.

D. Repository Retrieval and Memory

Repository-level generation benefits from retrieving relevant files and functions [22, 23]. Retrieval-augmented generation more broadly conditions model outputs on external evidence [24]. Episodic memory may transfer useful patterns across tasks, as in Reflexion [20], but evaluation-time memory creates contamination risks when benchmark instances influence later instances. In our primary evaluation, memory is empty and reset for every task. Cross-task memory is treated as a separate experiment and may be populated only from a documented, disjoint training set.

III. AGENTFORGE

A. Problem Definition and Information Boundary

For each task i , the agent receives an issue description q_i and a repository checkout R_i^0 at the benchmark base commit. The agent produces a patch p_i that transforms the repository into $R_i^* = R_i^0 \oplus p_i$. The final patch is successful only if the official benchmark harness accepts it.

The following fields are prohibited from all agent-visible inputs and retrieval indexes during inference:

- the reference or gold patch;
- file paths extracted from the reference patch;
- benchmark grading labels or expected outcomes;
- FAIL_TO_PASS and PASS_TO_PASS identifiers;
- data derived from other evaluation instances.

The agent may inspect only the base repository, the issue statement, command outputs produced during its own run, and optional memory built from a disjoint training set.

B. Roles and Shared State

AGENTFORGE contains five role-specific agents coordinated by a deterministic Orchestrator:

a) Planner.: The Planner converts the issue description and retrieved repository context into a structured repair plan. The plan may identify investigation steps and candidate validation commands, but it cannot waive the execution invariant.

b) Coder.: The Coder proposes a minimal patch against the current repository state. Existing files are edited through unified diffs where possible.

c) Tester.: The Tester may construct auxiliary tests or reproduction scripts from the issue description and inspected code. These artifacts are used for diagnosis only and are labeled as model-generated evidence.

d) Debugger.: The Debugger receives the current patch, command, exit status, standard output, and standard error. It proposes a revised patch targeted at the observed failure.

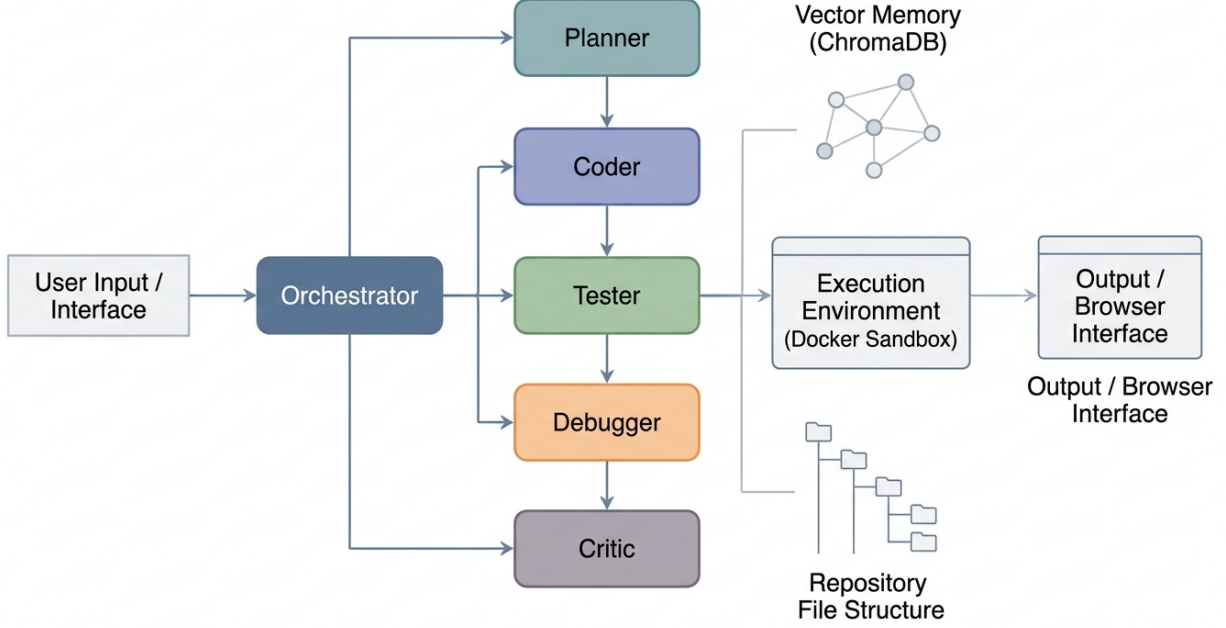


Fig. 1. Conceptual architecture of AGENTFORGE. For benchmark evaluation, the repository is initialized from the base commit, cross-instance memory is disabled, and all state-changing edits are followed by execution in the same task environment. The figure is architectural; the official benchmark harness independently evaluates the final patch.

e) Critic.: The Critic reviews the issue, cumulative diff, executed commands, and recorded outcomes. It may stop the search or request another repair iteration, but it cannot declare benchmark success. Only the official harness determines resolution.

All roles operate on one mutable repository state and one append-only execution history. This avoids parallel agents silently reasoning over inconsistent versions of the codebase.

C. Post-Edit Execution Invariant

Let R_t denote the repository state at step t . A state-changing action a_t applies a patch and produces R_{t+1} . The Orchestrator maintains a Boolean flag x_t indicating whether execution is required. The invariant is

$$R_{t+1} \neq R_t \Rightarrow x_{t+1} = 1, \quad (1)$$

and no subsequent state-changing action is permitted while $x = 1$. The flag is cleared only after a command is executed in the same repository environment and its outcome is appended to the history. Thus, execution is enforced by control flow rather than by hoping that the Planner schedules a Tester step.

Two properties of Algorithm 1 are worth stating explicitly, because both were absent from our earlier design. First, a rejected patch leaves R unchanged and therefore owes no execution; only edits that actually mutate the repository set x . Second, the returned patch is $\text{DIFF}(R^0, R)$ — read from the repository — rather than a diff extracted from model prose. An agent that describes an edit without performing one yields an empty patch, which the harness scores as unresolved. Section III-G describes the tests that hold both properties in place.

D. Execution Signals and Test Separation

The system distinguishes three forms of evidence:

- 1) **Repository commands**, such as targeted project tests, linters, static checks, or reproduction scripts available from the base repository.
- 2) **Agent-generated tests**, which are diagnostic hypotheses and may be wrong, incomplete, or overfitted to the candidate patch.
- 3) **Official benchmark tests**, which are hidden from the agent and used only by the independent evaluation harness.

A successful auxiliary command indicates that a candidate survived a particular check; it does not establish ground-truth correctness. The manuscript therefore uses the term *execution feedback*, not *ground-truth feedback*, for agent-visible runtime outcomes.

Treating generated tests as diagnostic rather than authoritative is a claim about their quality, so it is measured rather than asserted. The protocol is pre-registered here and implemented as a separate program.

For each instance in a development split, the Tester receives exactly the leakage-free input it receives during repair — issue statement, retrieved context from the base checkout, and nothing else — and produces a reproduction script. The script is run twice in the instance container: first on the base checkout, where the defect is present, and then on the same checkout with the reference patch applied, where it is fixed. A script is counted as *discriminating* only when it fails on the buggy tree and passes on the fixed one. The three failure modes are reported separately rather than folded into a single rate, because

Algorithm 1 AgentForge repair loop. The execution flag x is owned by the orchestrator, not by the plan; line 5 is an assertion that fails the run rather than a branch the Planner can influence.

Require: Issue q , base repository R^0 , budget B
Ensure: Model patch p and trajectory H

```

1:  $R \leftarrow R^0$ ;  $H \leftarrow []$ ;  $x \leftarrow 0$ ;  $\text{memory} \leftarrow \emptyset$ 
2:  $C \leftarrow \text{RETRIEVE}(q, R^0)$  {base checkout only}
3:  $P \leftarrow A_{\text{plan}}(q, C)$ ;  $s \leftarrow A_{\text{test}}(q, C)$ 
4: while BUDGETREMAINING( $B$ ) do
5:   assert  $x = 0$  {no edit while execution is owed}
6:    $a \leftarrow \text{debugger}$  if  $H$  holds a failure else  $\text{coder}$ 
7:    $\Delta \leftarrow A_a(q, C, P, H)$ 
8:    $(ok, m) \leftarrow \text{APPLYPATCH}(R, \Delta)$ 
9:   if  $\neg ok$  then
10:     $H.\text{APPEND}(a, \Delta, m)$  { $R$  unchanged, so no execution is owed}
11:   continue
12: end if
13:  $x \leftarrow 1$  { $R$  changed}
14:  $c \leftarrow \text{SELECTCOMMAND}(q, s)$  {never reads graded test ids}
15:  $(z, o, e) \leftarrow \text{EXECUTE}(R, c)$ 
16:  $H.\text{APPEND}(a, \Delta, c, z, o, e)$ ;  $x \leftarrow 0$ 
17: if  $z = 0$  then
18:    $v \leftarrow A_{\text{critic}}(q, \text{DIFF}(R^0, R), H)$ 
19:   if  $v = \text{STOP}$  then
20:     break
21:   end if
22: end if
23: end while
24: assert  $x = 0$  {loop cannot exit owing an execution}
25:  $p \leftarrow \text{DIFF}(R^0, R)$  {repository state, not parsed model output}
26: return  $(p, H)$ 
```

they have different causes and different consequences: scripts that pass on both trees are vacuous and provide the agent with false reassurance; scripts that fail on both are typically broken imports or a misread of the issue, and mislead the Debugger into repairing the wrong thing; scripts that never execute indicate an environment failure rather than a modelling failure.

Two properties of this measurement matter. First, it reads the reference patch, which is legitimate because it is measurement code rather than agent code, and because the patch is applied only after the script has been generated and run on the buggy tree — the Tester never observes it. Second, it is run on a split disjoint from the one used for headline results. Measuring auxiliary-test quality on the evaluation split and then reporting repair performance on the same instances would be a form of tuning against the test set, even though no reference patch reaches the agent.

The disjoint-development auxiliary-test discrimination study has not yet been completed, so no discrimination rate or failure-mode count is reported in this version.

E. Repository Retrieval and Episodic Memory

Retrieval operates inside the task container over the base checkout only, and is specified completely so that it can be reimplemented. Query terms are identifiers extracted from the issue statement: backtick-quoted spans, dotted paths, CAMELCASE names, and snake_case names of at least four characters, minus a stop list of common Python keywords. Terms are ranked by rarity within the issue, on the assumption that a name mentioned once discriminates better than a word repeated throughout the report, and the top 12 are retained.

Candidate files are those matching a query term under `grep` across tracked `.py` files, excluding `test/`, `tests/` and `testing/` directories. A file scores one point per distinct matching term plus three points per term appearing in a `def` or `class` declaration rather than an arbitrary line; the top eight files are selected. Each is split into chunks of at most 120 lines with 20 lines of overlap, chunks are scored by the same term count, and the best 12 are emitted in descending score order until a 24,000-character ceiling is reached. A chunk that would cross the ceiling is dropped rather than truncated mid-line. No query, index entry, or ranking signal derives from the reference patch.

The primary benchmark condition uses no cross-instance episodic memory. The evaluation launcher constructs a fresh orchestrator and a fresh container for every instance, so no plan, retrieved chunk, trajectory, or Critic judgment survives from one task to the next; there is no persistent store to initialize, and none to contaminate. This is a change from our earlier implementation, which reused a single orchestrator across the benchmark and wrote successful outputs into a process-level store, allowing instance n to condition on instances $1..n-1$. Any future memory-enabled experiment must draw on a frozen store built from a disjoint split and be reported separately.

F. Sandbox and Repository Environment

Interactive commands run inside the official SWE-BENCH instance image for the task, with the repository checked out at the base commit under `/testbed` and project dependencies preinstalled in the image’s conda environment. This matters more than it may appear: a generic Python image cannot import the project under repair, so commands issued against it produce import errors rather than information about the patch. Because dependencies are preinstalled, networking stays disabled throughout interaction.

Using the official instance image is also what makes the arrangement leakage-free. These images ship the repository at the base commit *without* the test patch applied — the harness applies it at grading time — so the agent can run the project’s own suite without ever seeing the tests it will be graded on. The container is created fresh per instance, reset to the base commit at startup, and force-removed afterwards.

Docker reduces accidental host interference but is not, by itself, a complete security boundary. Containers run with a 4GB memory cap, a two-CPU quota, a 512-process limit, networking disabled, and a per-command timeout enforced by `timeout(1)`; they retain the image’s default user and the Docker default capability and seccomp profiles. We

make no claim of production-grade isolation, and note that a determined adversarial payload is outside the threat model this configuration addresses.

G. Verifying the Invariants

Design claims of this kind are easy to assert in prose and easy to violate in code, so both invariants are checked by tests that ship with the artifact and run in continuous integration.

The information boundary is covered by a suite that builds an instance whose reference patch and graded test identifiers are distinctive strings, constructs the agent-visible projection, and asserts that no line of the reference patch, no graded test identifier, and no file path appearing only in the reference patch is present anywhere in the projection. Because the boundary concerns values rather than vocabulary, the runtime check compares against patch *content* — a prompt may contain the word “patch,” but not a line of the reference fix. Two further tests assert that the method that previously mined file paths from `problem_statement + patch` is absent, and that the adapter exposes no local grading function, since that is how an unofficial grader entered the pipeline in the first place.

The execution invariant is covered by a suite that drives the loop with a scripted model and a recording environment, then asserts over the resulting event sequence that every applied edit is immediately followed by an execution, that no two edits are ever adjacent, and that the loop cannot terminate with an edit outstanding. A complementary test asserts that a patch which fails to apply owes no execution, and another asserts that the optional-execution arm *can* skip execution — without which the ablation in Section V would compare two identical systems. A final test asserts that the run record contains no correctness field, so that the Critic’s decision to stop cannot be mistaken for a resolution verdict.

We report these tests not as a contribution but as the minimum evidence a reader needs to believe the architecture description, given that our earlier implementation diverged from its own specification in exactly these two places.

IV. EXPERIMENTAL DESIGN

A. Benchmark

We evaluate on SWE-BENCH Lite, a 300-instance subset of real GitHub issues and repository snapshots [2]. Each task is considered resolved only when the official harness applies the generated patch and all required regression and repair checks succeed. Patch-application rate is reported separately and, by definition, cannot be lower than resolution rate.

B. Leakage-Free Evaluation Protocol

The boundary is enforced structurally rather than by convention. Prediction and grading are separate programs. The prediction program reads an allowlist of four instance fields — identifier, repository, base commit, and problem statement — and constructs the agent-visible prompt from those alone; a runtime check compares every prompt and every retrieved context block against the instance’s reference patch, test patch, and graded test identifiers, and aborts the run on a match.

Issue-thread hint text is excluded by default, since on many instances it names the offending file, which is a softer form of the same localization leak; it is available only behind an explicit flag for a labelled ablation.

The prediction program contains no grading logic at all. It writes patches in the official prediction format and stops. The grading program invokes the official harness, which builds the instance environment, applies the test patch, runs the declared `FAIL_TO_PASS` and `PASS_TO_PASS` tests with the correct per-repository log parser, and emits a report that we read without modification. Because the two programs do not share a process, the agent cannot observe grading outcomes for its own task or revise its patch afterwards.

This replaces the arrangement in the rejected version, in which context files were derived by extracting file paths from the concatenation of the problem statement and the reference patch, and grading was performed by a local routine that cloned each repository, ran a bare `pytest`, and inferred resolution from stdout. That routine never applied the test patch, so the `FAIL_TO_PASS` tests it purported to check were not present in the tree it graded. We report this plainly because the corrected protocol is only meaningful against a clear account of what it corrects.

The release will include:

- a frozen repository commit and container image identifiers;
- the exact benchmark version and instance list;
- the version of the official harness used for grading, together with the operating system and container-runtime versions of the machine that produced the report;
- the subset file, including its seed and per-repository counts, for every result reported on a subset;
- task-level prediction JSONL files;
- task-level trajectories, token counts, execution counts, wall-clock time, and API cost;
- scripts that reproduce every table and confidence interval.

We record the harness and runtime versions rather than only the benchmark version because instance images are resolved by name from a public registry and the harness’s image-naming convention has changed across releases. A stale or mistaken image reference does not fail loudly: the run proceeds, every instance fails during environment setup, and the harness records the outcome as an error rather than a non-resolution. Aggregated without care, that is indistinguishable in the headline number from a system that solved nothing, so the grading environment is part of the reported configuration rather than an implementation detail.

C. Controlled Comparisons

The primary factorial comparison separates role decomposition from execution scheduling. All configurations use the same base model, issue statement, repository access, tool set, context-window limit, token budget, API-cost ceiling, wall-clock limit, and maximum number of environment executions.

The ReAct arm deserves comment, because our earlier implementation of it could not support the comparison we drew from it. That version had no repository access — its file-reading action always replied that no repository was available

TABLE I

CONTROLLED CONFIGURATIONS. “FORCED EXECUTION” MEANS THE ORCHESTRATOR BLOCKS FURTHER EDITS UNTIL A POST-EDIT COMMAND HAS RUN AND ITS OUTCOME HAS BEEN RECORDED. THE FOUR ROWS ABOVE THE RULE FORM THE PRIMARY FACTORIAL; REACT IS A MATCHED SINGLE-AGENT COMPARISON; THE SINGLE CALL IS A FLOOR.

Configuration	Role structure	Repository	Tools	Iterative edits	Forced execution	Matched budget
Single agent, optional execution	Single	Yes	Yes	Yes	No	Yes
Single agent, forced execution	Single	Yes	Yes	Yes	Yes	Yes
Multi-agent, optional execution	Five roles	Yes	Yes	Yes	No	Yes
AGENTFORGE	Five roles	Yes	Yes	Yes	Yes	Yes
ReAct	Single	Yes	Yes	Yes	No	Yes
Single call, no tools	Single	No	No	No	No	No

— and its test action returned a fixed string reporting that three tests had passed, regardless of what the agent had written. It was therefore an agent with no environment being told that its work succeeded, and any margin measured against it reflected that handicap rather than any property of AGENTFORGE. The current ReAct arm runs in the same instance container with the same tools, retrieval, model and budget; only the control structure differs.

The single-call arm has no tools, no repository, and no execution. It is reported as a floor and never used to isolate the effect of decomposition or execution, because it differs from AGENTFORGE in every respect simultaneously, so a difference against it attributes to nothing in particular.

Published results from SWE-agent, OpenHands, MAGIS and other systems appear in a separate contextual table, since their models, budgets, prompts and environments differ from ours. We do not apply paired significance tests to aggregate results taken from other publications.

D. Budget and Implementation Controls

One control deserves particular emphasis. Our framework supports per-role model routing, and its defaults assigned different providers to different roles. Any configuration run under those defaults is a mixture of models, and reporting it under a single model name — as the rejected version did — misattributes to architecture what may be an effect of model composition. Every configuration compared here is therefore run with one model across all five roles, set explicitly at the command line, and the resolved model name is recorded per call in the released trajectories. Per-role routing remains available but is reported, if used at all, as a separate labelled experiment.

All configurations additionally share one budget object with identical caps on iterations, environment executions, API cost, and wall-clock time, and per-call token and cost accounting is recorded from provider usage fields rather than estimated. For every locally rerun configuration, report:

- exact model name and API version;
- sampling parameters and retry policy;
- maximum input and output tokens;
- maximum API cost per task;
- maximum wall-clock time and execution attempts;
- repository commands available to the agent;
- number and size of retrieved context chunks;
- whether auxiliary tests are enabled.

When an API does not guarantee deterministic sampling, repeated runs should be treated as repeated stochastic trials rather than claimed as exact replication from a seed alone.

E. Metrics and Statistical Analysis

The primary metric is official task resolution: the fraction of submitted instances the SWE-BENCH harness marks resolved. *Patch rate* is the fraction of instances for which the system emitted a non-empty patch. Since a non-empty patch is necessary for resolution, patch rate is at least resolution rate by construction; the analysis code asserts this rather than merely stating it. The rejected version of this paper reported a patch rate below its resolution rate, which was not a property of the system but an artifact of a grading path that no longer exists.

Secondary metrics are token usage, API cost, wall-clock time, model calls, environment executions, and cost per resolved task. Uncertainty in resolution rate is reported with an exact Clopper–Pearson interval rather than a normal approximation, which is poorly behaved at the counts involved. Cost is reported as median and interquartile range alongside the mean, because agent trajectories are heavy-tailed: a handful of instances that exhaust the iteration budget dominate the mean.

McNemar’s test is applied only to per-instance outcome vectors from two configurations run locally on the same instance list. The analysis code takes two run directories, intersects their instance sets, forms the discordant pairs, and computes an exact binomial test; it raises rather than returning a p -value when the intersection is empty. There is deliberately no code path that accepts two aggregate rates, because that is what our earlier version did when comparing against published numbers, and the resulting test statistic was not defined. Comparisons against systems we did not rerun are therefore reported without significance tests.

V. RESULTS

This section reports only values generated by the corrected implementation. Official resolution is reported only when a saved official SWE-BENCH harness report exists; generation-stage patch and resource statistics are taken from the released task-level run records.

A. Main Results

TABLE II

SWE-BENCH LITE RESULTS AND GENERATION-STAGE STATISTICS ON THE SEEDED 50-INSTANCE SUBSET. OFFICIAL RESOLUTION IS REPORTED ONLY WHERE A SAVED OFFICIAL HARNESS REPORT IS AVAILABLE; UNGRADED CONFIGURATIONS ARE MARKED "NOT GRADED." THE AGENTFORGE CONFIDENCE INTERVAL IS EXACT CLOPPER-PEARSON. PATCH RATE, MEAN COST, AND MEAN EXECUTIONS ARE COMPUTED PER ATTEMPTED TASK FROM THE RELEASED RUN RECORDS.

Method	Resolved	Resolve rate [95% CI]	Patch rate	Mean cost	Mean execs
Single call, no tools	Not graded	Not graded	18.0%	\$0.0005	0.00
ReAct	Not graded	Not graded	0.0%	\$0.0056	0.10
Single, optional execution	Not graded	Not graded	14.0%	\$0.0022	0.00
Single, forced execution	Not graded	Not graded	18.0%	\$0.0024	0.20
Multi-agent, optional execution	Not graded	Not graded	28.0%	\$0.0051	0.74
AGENTFORGE	1/50	2.0% [0.05%, 10.65%]	24.0%	\$0.0056	0.60

The primary analysis should answer two paired questions. First, does forced post-edit execution improve performance when the role structure is held constant? Second, does role decomposition improve performance when execution scheduling is held constant? Any interpretation must account for confidence intervals, paired outcomes, and resource use rather than percentage differences alone.

B. Published Results in Context

Table III summarizes published SWE-BENCH Lite results from prior work for contextual reference. It is deliberately a separate table, and the separation is not cosmetic. Every entry differs from ours in several respects at once — base model, token and cost budget, prompt design, scaffold, retrieval strategy, and in several cases the harness version and the date at which the split was scored. A difference measured against any of them is therefore not attributable to role decomposition, to enforced execution, or to any other single property of the system. We report no significance tests against these numbers, and the analysis code has no path that accepts two aggregate rates, because computing a paired statistic from figures produced on different instance-level outcomes is undefined — the earlier version of this paper did exactly that.

Three properties of the table are worth stating explicitly. First, the model column matters as much as the system column: the same scaffold reported at 18.00% with GPT-4 and at 56.67% with a 2025-generation model, so a large fraction of the visible spread reflects base-model progress rather than architectural difference. Second, the leaderboard entries are self-reported by their authors and were not reproduced by us; we quote them as context with an access date rather than as verified measurements, and we mark those without an accompanying publication as such.

Third, the cost column is reported where the source reports it, but it does not support the comparison that Table IV supports. Those figures were produced under different provider prices, different token accounting, and different conventions about whether retried or abandoned attempts are counted; a lower number there may reflect cheaper tokens in a given quarter rather than a more economical method. Accordingly, we do not rank these systems by cost effectiveness from this table. Additional test-time computation is not self-justifying, which is the reason this paper holds budgets equal within its own comparisons rather than reasoning across published figures.

TABLE III

PUBLISHED SWE-BENCH LITE RESULTS FROM PRIOR WORK, FOR CONTEXT ONLY. ENTRIES DIFFER IN MODEL, BUDGET, SCAFFOLD AND HARNESS VERSION SIMULTANEOUSLY, SO NO DIFFERENCE IS ATTRIBUTED TO A SINGLE CAUSE AND NO SIGNIFICANCE TEST IS APPLIED. LEADERBOARD FIGURES ARE SELF-REPORTED AND NOT INDEPENDENTLY REPRODUCED. COST IS REPRODUCED AS REPORTED BY EACH SOURCE; A DASH DENOTES A FIGURE THE SOURCE DOES NOT REPORT.

System	Model	% Resolved	Cost / inst.	Source
<i>Non-agentic pipeline</i>				
Agentless	GPT-4o	32.00	~\$0.70	[25]
<i>Single-agent scaffolds</i>				
SWE-agent	GPT-4	18.00	~\$2.51	[3]
SWE-agent	Claude 4 Sonnet	56.67	—	[26]
AutoCodeRover	GPT-4	19.00	~\$0.45	[5]
AutoCodeRover-v2	GPT-4o	30.67	—	[26]
OpenHands + CodeAct v2.1	Claude 3.5 Sonnet	41.67	—	[4, 26]
<i>Multi-agent systems</i>				
MAGIS	GPT-4	13.94	—	[6]
CodeR	—	28.33	—	[26]
MarsCode Agent	—	39.33	—	[26]
ExpeRepair	Claude 4 Sonnet	60.33	—	[26]
Refact.ai Agent	—	60.00	—	[26]

TABLE IV

RESOURCE USE PER ATTEMPTED TASK FOR THE FOUR FACTORIAL ARMS IN THE SUPPLIED DIAGNOSTIC RUN. VALUES ARE COMPUTED FROM THE 50 TASK-LEVEL TRAJECTORIES; THE SAME 22 INSTANCES WERE ABORTED BY THE LEAKAGE GUARD IN ALL FOUR ARMS AND THEREFORE CONTRIBUTE ZERO POST-ABORT MODEL USAGE. COST IS MEDIAN WITH INTERQUARTILE RANGE.

Method	Tokens	Calls	Executions	Median cost (IQR)
Single, optional execution	33,760	4.5	0.0	\$0.0006 (\$0.0000–\$0.0030)
Single, forced execution	27,953	3.7	0.2	\$0.0014 (\$0.0000–\$0.0041)
Multi-agent, optional execution	48,533	5.5	0.7	\$0.0023 (\$0.0000–\$0.0088)
AGENTFORGE	44,096	5.3	0.6	\$0.0037 (\$0.0000–\$0.0092)

C. Equal-Budget and Cost-Effectiveness Analysis

A method that uses more computation may still be useful, but the manuscript should describe the tradeoff directly. Report cost per resolved task and resolution at fixed cost ceilings rather than characterizing a more expensive system as more sample efficient without evidence.

D. Ablation Study

Agent-removal ablations must specify exactly how the pipeline changes. Removing a role should not silently alter the number of calls, tool permissions, execution opportunities, or context budget. A preferred design reallocates the removed role’s budget to the remaining configuration or reports both fixed-architecture and equal-budget ablations.

Because the grid multiplies the cost of every instance by the number of conditions, ablations are reported on a subset rather than on all 300 instances. The subset is a stated object rather than a by-product of a task limit. SWE-BENCH splits are grouped by repository and are not shuffled, so truncating to the first N instances is not a sample: it over-represents whichever projects sort first and can omit others entirely. We instead allocate the N slots across repositories in proportion to each repository’s share of the split, using the largest-remainder method so the per-repository counts sum exactly to N , and draw within each repository under a fixed seed after sorting the candidate identifiers, so the selection does not depend

TABLE V

STATUS OF THE PRE-SPECIFIED ABLATION RUNS IN THE SUPPLIED ARTIFACT. BECAUSE MOST ABLATIONS HAVE NO OFFICIAL HARNESS REPORT AND SEVERAL WERE INTERRUPTED BY PROVIDER API FAILURES, RESOLUTION EFFECTS AND PAIRED DIFFERENCES ARE NOT REPORTED. “COMPLETED” COUNTS TRAJECTORIES THAT REACHED THE REPAIR LOOP WITHOUT A TOP-LEVEL RUN ERROR.

Configuration	Completed	Guard aborts	Provider failures	Non-empty patches	Official grading
Full AGENTFORGE	28	22	0	12/50	Diagnostic report only
Without Planner	0	22	28	0/50	Not graded
Without Tester	0	22	28	0/50	Not graded
Without Debugger	19	22	9	11/50	Not graded
Without Critic	28	22	0	14/50	Not graded
Without repository retrieval	0	16	34	0/50	Not graded
Without generated tests	0	22	28	0/50	Not graded

on dataset iteration order. The seed and the resulting per-repository counts are written alongside the instance list, so a reviewer can regenerate the identical subset and verify it. Selection reads only the instance identifier and repository name; it never inspects a reference patch, and so cannot bias the subset toward instances of any particular difficulty. Every condition is intended to run on the same instance list, which is required for the paired differences planned for Table V. In the supplied artifact, however, several ablation runs are incomplete because of provider API failures; Table V therefore reports run status rather than effect estimates.

E. Failure Analysis

Failure analysis is conducted on a stratified sample of unresolved instances, drawn by the same seeded procedure used for the evaluation subset so that the sample is reproducible and not chosen after inspecting outcomes. Two annotators code each instance independently from the released trajectory — the retrieved context, the diffs attempted, the commands executed and their output — without access to the reference patch, which would make localization judgements trivially post hoc. Coding is multi-label, because a run that mislocalizes and then exhausts its budget exhibits both, and forcing a single cause would understate the more common of the two.

The codebook fixes seven categories. *Localization error*: no edit touches a file modified by the reference patch. *Incomplete semantic repair*: the correct file is edited but the change does not implement the required behaviour. *Invalid patch*: the diff never applies, whether malformed, misaligned, or addressed to a path that does not exist. *Environment failure*: the run is defeated by dependency, build, or container problems rather than by the repair task. *Auxiliary-test overfitting*: the agent’s generated test passes while the official tests do not, so runtime evidence actively pointed the wrong way. *Low-diversity repetition*: successive attempts are substantially the same edit, indicating the loop stopped exploring. *Budget exhaustion*: the run terminates on the iteration, execution, cost, or wall-clock cap with work still outstanding.

Two of these categories are diagnostic for the mechanism this paper argues for, and are worth separating for that reason. Auxiliary-test overfitting is the failure mode that enforced execution is most likely to *introduce*: an agent that must act on runtime evidence can be misled by evidence that is wrong. Low-diversity repetition is the mode it is most likely to *reveal*, since an agent that never executes cannot discover that its edit

changed nothing. Reporting them separately therefore tests the mechanism rather than only counting its successes.

Agreement is reported as Cohen’s κ over the multi-label coding, computed per category, before adjudication; disagreements are then resolved by discussion and the adjudicated counts reported. Reporting κ before adjudication rather than after is deliberate, since post-adjudication agreement is near unity by construction and says nothing about whether the codebook is applicable.

The adjudicated failure-analysis study has not yet been completed; consequently, per-category counts, representative examples, and Cohen’s κ values are not reported in this version.

VI. THREATS TO VALIDITY

a) *Construct validity*.: Official SWE-BENCH resolution is a binary measure of benchmark success, not a complete measure of software quality, maintainability, security, or developer productivity. Agent-generated tests provide partial diagnostic evidence and may be correlated with the candidate patch.

b) *Internal validity*.: Agent systems are sensitive to prompts, model versions, context truncation, execution budgets, and transient API behavior. The revised study controls these factors within local comparisons and releases task-level trajectories. It does not infer causal effects from comparisons with published aggregate scores produced under different conditions.

c) *External validity*.: SWE-BENCH Lite consists primarily of Python repositories and may not represent other languages, build systems, or industrial workflows. A single benchmark also cannot establish general superiority across software engineering tasks.

d) *Memory and contamination*.: Cross-task memory can leak evaluation information even without explicit reference patches. The primary study therefore resets memory per instance. Any memory-enabled experiment must use a frozen, documented, disjoint source corpus.

e) *Security*.: Resource-limited containers reduce but do not eliminate risks from untrusted generated code. The system is an experimental research artifact and should not be treated as a production security boundary.

VII. CONCLUSION

We presented a revised formulation of AGENTFORGE as a role-structured framework for repository-level software repair. The system separates planning, editing, auxiliary test generation, debugging, and review, while an orchestrator-enforced invariant requires execution after every repository-changing edit. The paper deliberately distinguishes runtime evidence available to the agent from hidden benchmark correctness and specifies a leakage-free, budget-controlled evaluation using the official SWE-BENCH harness.

We state plainly what this version does not claim. The numerical results of the earlier version of this work could not be reproduced by the implementation that was said to have produced them, and they are not carried forward in any form. Nothing in this manuscript asserts a resolution rate, a margin over a baseline, or an ablation effect that is not read directly from an official harness report generated by the released code.

Where such a report does not yet exist, the corresponding entry is marked as outstanding rather than estimated, and the tables state how many configurations have been graded.

In the available official grading, AGENTFORGE resolves 1 of 50 submitted tasks (2.0%, exact 95% Clopper–Pearson CI: 0.05%–10.65%) and emits a non-empty patch on 24.0% of attempted tasks. Because the matched control configurations do not yet have saved official harness reports, these observations do not establish an advantage from role decomposition or forced execution, and the incomplete ablation runs do not support effect estimates.

The broader lesson is methodological, and it survives whatever the rerun shows: claims about agent architecture require transparent information boundaries, identical evaluation infrastructure, paired outcomes, and explicit accounting for test-time computation. An architecture that appears to help under oracle localization, an inflated baseline, or a grader that never applies the benchmark’s own tests has not been shown to help at all.

APPENDIX A AGENT PROMPTS

These are the verbatim system prompts of the evaluated implementation, reproduced here so that the paper is self-contained if the repository is unreachable. They differ from those in the rejected version, which described roles operating on free-floating code strings rather than on a repository. The user message accompanying each is assembled from the issue statement, retrieved context, triage output, and execution history as described in Section III.

A. Planner

You are a senior software engineer triaging a bug report against an unfamiliar repository. Given the issue and retrieved repository context, state in at most five sentences: the most likely responsible file(s), the suspected root cause, and one shell command that would demonstrate the bug. You are not given the fix. Reason only from the issue text and the code shown.

B. Coder

You are an expert software engineer. Produce a minimal unified diff in git format that fixes the described issue. Use exact paths relative to the repository root. Include at least three lines of context per hunk. Do not modify test files. Return ONLY the diff inside ``diff fences.

C. Tester

You write a short reproduction script for a bug report. Return ONLY a runnable Python script that exercises the described behaviour and exits non-zero when the bug is present. This is a diagnostic aid, not a specification of correctness.

D. Debugger

You are an expert debugger. You are given a patch, the command that was run, its exit status and its output. Diagnose why it failed and return a corrected minimal unified diff against the ORIGINAL files, in git format, inside ``diff fences. Return ONLY the diff.

E. Critic

You are a senior code reviewer. Given an issue, a cumulative diff and the commands executed with their outcomes, reply “STOP” if the change plausibly resolves the issue and the evidence supports stopping, or “CONTINUE: <what to investigate next>”. You are not deciding benchmark correctness.

F. Single-Agent Control

You are an expert software engineer resolving an issue in a repository. Given the issue, repository context and the history of your own edits and command outputs, return the next minimal unified diff in git format inside ``diff fences, or the single word DONE if the issue is resolved.

This last prompt defines the single-role arm of the factorial. It has the same repository access, the same tools, the same retrieval, and the same budget as AGENTFORGE; only the decomposition differs. It is not the no-tool, single-call baseline, which is reported separately as a lower bound.

APPENDIX B REPRODUCIBILITY CHECKLIST

The full pipeline is three commands. Prediction and grading are separate programs, and the second is the official harness:

```
python -m eval.swebench_runner --split lite \
    --config agentforge --model gpt-4o \
    --run_id af_lite_001

python -m eval.run_official_eval --run_id af_lite_001

python -m eval.analyze_results \
    --runs af_lite_001 single_forced_001 \
    --baseline single_forced_001 --latex
```

The `-config` flag selects a cell of the factorial (agentforge, multi_optional, single_forced, single_optional) or an ablation (no_retrieval, no_generated_tests); budget flags are shared across all of them. The artifact provides:

- frozen source commit and dependency lock files;
- the prompts in Appendix A as plain-text files, plus the prompt-construction code;
- benchmark version, split, and instance IDs;
- the leakage and invariant test suites of Section III-G, runnable in isolation;
- instance image identifiers resolved per task;
- the official harness invocation and the prediction JSONL it consumed;
- the unmodified harness report for every run;
- per-task trajectories with token, call, execution, latency, and cost records;
- scripts reproducing every table, interval, and paired test.

We note one limit on reproducibility that a seed does not remove: the provider APIs used here do not guarantee deterministic sampling even at temperature zero with a fixed seed. Repeated runs are therefore reported as repeated stochastic trials, not as replications.

REFERENCES

- [1] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, “Evaluating large language models trained on code,” 2021, codex / GitHub Copilot paper.
- [2] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. R. Narasimhan, “SWE-bench: Can language models resolve real-world github issues?” in *The Twelfth International Conference on Learning Representations*, 2024, iCLR 2024 Oral.
- [3] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, “Swe-agent: Agent-computer interfaces enable automated software engineering,” in *Advances in Neural Information Processing Systems*, vol. 37, 2024, neurIPS 2024.
- [4] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, M. Zhuge, J. Pan, Y. Song, B. Li, J. Singh *et al.*, “Openhands: An open platform for ai software developers as generalist agents,” 2024, primary open-source baseline for multi-agent SWE (also accepted as ICLR 2025 poster).
- [5] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury, “Autocoderover: Autonomous program improvement,” 2024, iSSTA 2024.
- [6] W. Tao *et al.*, “Magis: Llm-based multi-agent framework for github issue resolution,” 2024.
- [7] S. Khanzadeh, “Agentmesh: A cooperative multi-agent generative ai framework for software development automation,” 2025.
- [8] S. Hong *et al.*, “Metagpt: Meta programming for a multi-agent collaborative framework,” 2023.
- [9] C. Qian *et al.*, “Chatdev: Communicative agents for software development,” 2023.
- [10] Y. Li *et al.*, “Competition-level code generation with alphacode,” 2022.
- [11] R. Li *et al.*, “Starcoder: May the source be with you!” 2023.
- [12] B. Rozière *et al.*, “Code llama: Open foundation models for code,” 2023.
- [13] C. Le Goues, T. Nguyen, S. Forrest, and W. Weimer, “Genprog: A generic method for automatic software repair,” *IEEE Transactions on Software Engineering (TSE)*, vol. 38, no. 1, pp. 54–72, 2012.
- [14] C. Xia and L. Zhang, “Less is more: Summary of long code for repair,” in *Proceedings of the IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2022.
- [15] M. Jin *et al.*, “Inferfix: End-to-end program repair with llms,” in *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*, 2023.
- [16] Z. Fan *et al.*, “Automated program repair in the era of large pre-trained language models,” 2023.
- [17] X. Chen *et al.*, “Teaching large language models to self-debug,” 2023.
- [18] T. X. Olausson *et al.*, “Is self-repair a silver bullet for code generation?” 2023, justification for the Iterative Debug Loop in AgentForge.
- [19] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “React: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [20] N. Shinn *et al.*, “Reflexion: Language agents with verbal reinforcement learning,” 2023.
- [21] Q. Wu *et al.*, “Autogen: Enabling next-gen llm applications via multi-agent conversation,” 2023.
- [22] F. Zhang *et al.*, “Repocoder: Repository-level code completion,” 2023.
- [23] D. Shrivastava *et al.*, “Repofusion: Training code models on whole repositories,” 2023.
- [24] P. Lewis *et al.*, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” 2020.
- [25] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, “Agentless: Demystifying llm-based software engineering agents,” *arXiv preprint arXiv:2407.01489*, 2024.
- [26] SWE-bench Team, “SWE-bench leaderboard,” <https://www.swebench.com/>, 2026, community-submitted results; entries are self-reported and were not independently reproduced by us. Figures quoted as of 2026-08-09.